

Topic 11: Friend Functions and Friend Classes

11.1 What is a Friend?

- A friend function is a non-member function granted access to private and protected members of a class
- Declared inside the class with the friend keyword: friend ReturnType funcName(params);
- Friendship is explicitly granted by the class -- it is not inherited or transitive
- Use sparingly -- it breaks strict encapsulation. Prefer public interfaces when possible

11.2 Friend Functions

- Useful when an operation logically involves two different classes equally
- Stream operators (<<, >>) are the most common use case
- The friend declaration acts as a prototype -- the actual definition is outside the class

```
class Complex {
    double real, imag;
public:
    Complex(double r=0, double i=0) : real(r), imag(i) {}

    friend Complex operator+(const Complex& a, const Complex& b);
    friend ostream& operator<<(ostream& os, const Complex& c);
};

Complex operator+(const Complex& a, const Complex& b) {
    return Complex(a.real + b.real, a.imag + b.imag);
}

ostream& operator<<(ostream& os, const Complex& c) {
    os << c.real << " + " << c.imag << "i";
    return os;
}
```

11.3 Friend Classes

- An entire class can be declared a friend: friend class ClassName;
- All member functions of the friend class gain access to private members of the granting class
- Use case: a LinkedList class granting its Node class access to private fields for efficiency

```
class Engine {
    int horsepower;
    friend class Car;    // Car can access Engine's private members
public:
    Engine(int hp) : horsepower(hp) {}
};

class Car {
    Engine engine;
public:
    Car(int hp) : engine(hp) {}
    void showPower() { cout << engine.horsepower << " HP" << endl; }
};
```